```
  def self.grouped_for_concert(concert_id)
    return [] unless concert_id
    for_concert(concert_id).map(&:to_concert_h).group_by { |t| t[:row] }.values
  end
```

For our purposes, this means our channel expects a message that tells it to invoke the `add_to_cart` method with the same data that was passed to the original `fetch` endpoint, and it uses `ActionCable.server.broadcast` to send the same complete list of seat setup data back over the channel. This broadcast will go to all the clients subscribed to the channel, crucially including the client that sent the message in the first place.

On the client side, we need to subscribe to the channel. We also need to send a message to it when we click on a seat, and we need to respond to the resulting data. Mostly this involves a slight restructuring of the existing code.

In the `Venue` component, since we still need to do our data fetch in the `useEffect` hook but don't want to poll the server anymore and instead want to subscribe to an ActionCable channel, we need to add a call to create the subscription:

chapter_09/04/app/javascript/components/venue.tsx

```
import * as React from "react"
import VenueBody from "./venue_body"
import VenueHeader from "./venue_header"
import { createConsumer, Subscription } from "@rails/actioncable"

interface VenueProps {
  concertId: number
  rowCount: number
  seatsPerRow: number
}

export interface TicketData {
  id: number
  row: number
  number: number
  status: string
}
export type RowData = TicketData[]
export type VenueData = RowData[]
```